



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:	11 de Agosto de 2026
Hora de inicio:	9:00 - 10:40

Lenguajes Formales y De Programación B-

José Estuardo Orellana Leonardo

Iniciamos a las 09:04

Lenguajes



Lenguajes Formales y de Programación

Escuela de Ingeniería de Ciencias Y Sistemas

Facultad de Ingeniería

Universidad de San Carlos de Guatemala

Agenda



RECORDATORIOS

Foro 4



Lenguaje Natural



Lenguaje Formal



Lenguajes de Programación



Evolución de los Lenguajes de Programación



Paradigmas



Generaciones de los Lenguajes de Programación

Recursos

Link del Video



https://www.youtube.com/watch?v=xTLbG3_Rs_w

Dudas clase anterior ?

Tema visto: Estructura de Datos en Python y Lectura, Escritura de Archivos



COMPETENCIA QUE DESARROLLAREMOS

El estudiante diferencia los tipos de lenguajes (natural, formal y de programación), reconoce su evolución, paradigmas y generaciones, y los aplica en el desarrollo de soluciones tecnológicas.



LENGUAJE NATURAL

El lenguaje natural es una herramienta esencial para la comunicación humana. Nos permite compartir nuestros pensamientos, sentimientos y experiencias con otras personas. También nos permite aprender, enseñar, negociar, resolver conflictos y construir relaciones.

- Hablamos con nuestros amigos, familiares y colegas.
- Escribimos cartas, correos electrónicos y mensajes de texto.
- Leemos libros, artículos y periódicos.
- Vemos películas y programas de televisión.
- Escuchamos música.
- Utilizamos la Internet para comunicarnos con personas de todo el mundo.



LENGUAJE FORMAL

Un lenguaje formal es un lenguaje artificial que, a diferencia de los lenguajes naturales, no está centrado en la comunicación humana habitual, sino en la definición y aplicación de una serie de reglas específicas en un entorno específico. Consta de una serie de símbolos básicos, que se combinan en cadenas para formar expresiones complejas.

Los símbolos constituyen el alfabeto de un lenguaje formal, mientras que las cadenas son las palabras. Tanto la combinación de los símbolos como la interpretación de las cadenas se realiza de acuerdo con reglas rigurosas, establecidas de antemano, que no admiten excepciones; en conjunto, estas reglas conforman la gramática del lenguaje.

símbolo	nombre	lectura informal
$\wedge$	conjunción	'y'
$\vee$	disyunción	'o'
$\supset$	condicional	'Si... entonces...'
$\neg$	negación	'no'
$\forall$	cuantificador universal	'para todo'
$\exists$	cuantificador existencial	'existe'
$\approx$	identidad	'es (idéntico a)'



REGULAR EXPRESSION

:

/

(

[

\

w

\

s

]

)

+

|

[

0

-

9

]

?

)

TEST STRING

H

o

l

a

m

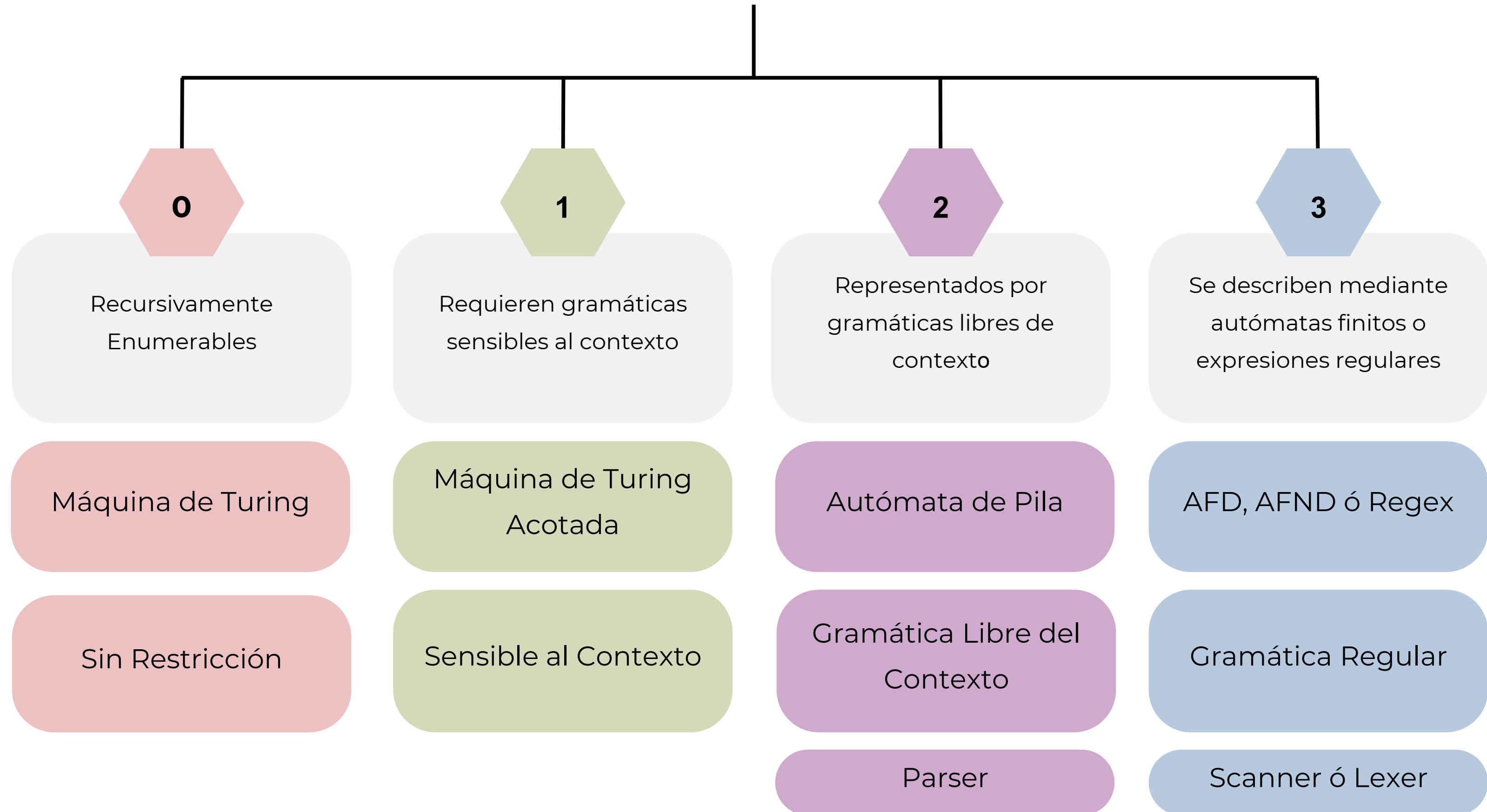
u

n

d

o

TIPOS DE LENGUAJES FORMALES



LENGUAJES DE PROGRAMACIÓN

Un lenguaje de programación es un conjunto de reglas que permiten a los desarrolladores comunicar instrucciones a las computadoras. Los lenguajes de programación son diseñados para ser concisos, fáciles de leer y fáciles de entender por las computadoras y para los humanos.

Los lenguajes de programación se utilizan para crear software mediante la escritura de algoritmos e instrucciones, que son programas informáticos que realizan tareas específicas, permitiendo la creación de software y sistemas computacionales.



EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN



El nacimiento de los lenguajes de programación

-  • **FORTRAN**: Primer lenguaje de alto nivel, diseñado para cálculos científicos.
-  • **COBOL**: Creado para aplicaciones empresariales y financieras.

La revolución del software

-  **PASCAL** Pascal: Popular en educación y software estructurado.
-  • **C**: Lenguaje fundamental para sistemas operativos (Unix) y aplicaciones.
-  • **SQL**: Introducido para gestionar bases de datos relacionales.

Internet y el auge de los lenguajes modernos

-  • **Python**: Lenguaje simple y poderoso para múltiples propósitos.
-  • **Java, JavaScript, PHP, Ruby**: Lenguajes clave para la web y aplicaciones empresariales.

Programación moderna y nuevas tendencias

-  • **Kotlin**: Alternativa moderna a Java para Android.
-  • **Swift**: Creado por Apple para desarrollo en iOS/macOS.
-  • **Rust**: Popular por su seguridad en sistemas de bajo nivel.



Estandarización y expansión

-  • **ALGOL**: Base para muchos lenguajes modernos, influyó en C y Pascal.
-  • **BASIC**: Facilita el aprendizaje de programación.

Orientación a objetos y más accesibilidad

-  • **C++**: Extensión de C con programación orientada a objetos.
-  • **Perl**: Potente para procesamiento de texto y scripting.

Desarrollo web y móvil

-  • **C#**: Creado por Microsoft para su ecosistema.
-  • **Scala**: Combina programación funcional y orientada a objetos.
-  • **Go**: Lenguaje de Google enfocado en rendimiento y concurrencia.

Inteligencia Artificial y nuevas tecnologías

-  • **Julia**: Crece en popularidad para computación científica y machine learning.
-  • **Python** sigue dominando en IA y Data Science, con crecimientos en lenguajes como **Rust** y **TypeScript**.
- 

GENERACIONES DE LENGUAJES DE PROGRAMACIÓN

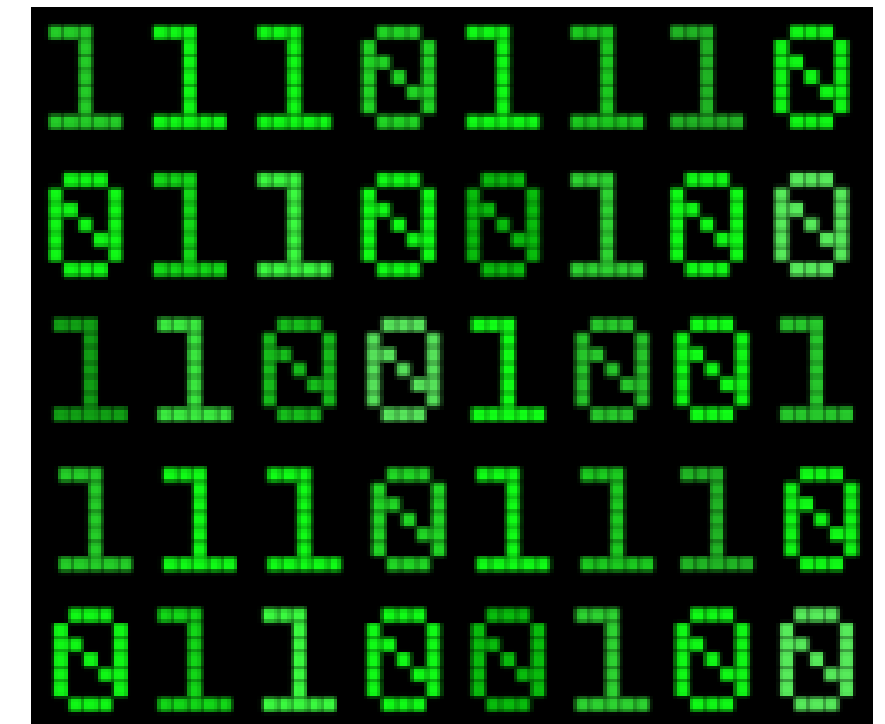


PRIMERA GENERACIÓN (1940 - 1950)

La primera generación de lenguajes de programación es el lenguaje de máquina, la forma más básica de programación.

El lenguaje de máquina consta de código binario, unos y ceros, que es ejecutado directamente por la CPU de una computadora. Es muy específico de la arquitectura del hardware de la computadora.

- **Características:** Puramente binario, difícil de leer y escribir para los humanos.
- **Ejemplos:** No hay ejemplos nombrados, ya que varía según el hardware.
- **Impacto:** Sentó las bases para la programación informática, pero era engorrosa y propensa a errores.



SEGUNDA GENERACIÓN (1950 - 1960)



El lenguaje ensamblador está un paso por encima del lenguaje de máquina y proporciona una ligera abstracción. Utiliza códigos o símbolos mnemotécnicos para representar instrucciones en lenguaje de máquina, lo que hace que sea un poco más fácil para los humanos comprenderlas y escribirlas.

- **Características:** Sigue siendo específico del hardware, pero más legible que el código binario.
- **Ejemplos:** NASM, MASM.
- **Impacto:** Hizo que la programación fuera más accesible, pero aún requería un profundo conocimiento del hardware.

```
.file "hola.c"
.section .rodata
.LC0:
.string "%d"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
movl $3, %edx
movl $2, %eax
addl %eax, %edx
;; movl $.LC0, %eax
;; movl %edx, %esi
```

TERCERA GENERACIÓN (1960 - 1970)

La tercera generación introdujo lenguajes de programación de alto nivel, que se acercan más a los lenguajes humanos y abstraen gran parte de los detalles específicos del hardware.

- **Características:** La sintaxis se parece a los lenguajes humanos, con construcciones poderosas que permiten operaciones complejas con declaraciones simples.
- **Ejemplos:** C, Fortran, COBOL, Pascal, BASIC.
- **Impacto:** Aumento significativo de la productividad y la accesibilidad en la programación. Permitió el desarrollo de sistemas de software complejos



CUARTA GENERACIÓN (1980 - 1990)

Los lenguajes de cuarta generación (4GL) son aún más abstractos y apuntan a una mayor productividad y simplicidad en la resolución de problemas específicos.

Se utilizan a menudo en consultas de bases de datos, generación de informes y manipulación de datos.

- **Características:** Centrarse en reducir el esfuerzo de programación y acercarse al lenguaje natural.
- **Ejemplos:** SQL, MATLAB, SAS, Visual Basic.
- **Impacto:** Mayor productividad en dominios específicos, como el análisis de datos y la gestión de bases de datos.



QUINTA GENERACIÓN (1980 - Actualidad)

Los lenguajes de quinta generación se centran en la resolución de problemas utilizando restricciones en lugar de algoritmos. Se utilizan en el desarrollo de inteligencia artificial, sistemas expertos y comprensión del lenguaje natural.

- Características: Uso de paradigmas de lógica y programación declarativa.
- Ejemplos: Prolog, Lisp, Mercurio.
- Impacto: Impulsó el desarrollo de IA y sistemas complejos de resolución de problemas.



PARADIGMAS DE PROGRAMACIÓN

Un paradigma de programación es un conjunto de principios y prácticas que se utilizan para diseñar y escribir programas informáticos. Los paradigmas de programación se pueden utilizar en combinación para crear programas más complejos y poderosos.

- **Programación imperativa:** este paradigma se basa en la idea de dar instrucciones a la computadora paso a paso.
- **Programación orientada a objetos:** este paradigma se basa en la idea de crear objetos que representan conceptos del mundo real.
- **Programación funcional:** este paradigma se basa en la idea de usar funciones para representar el comportamiento de un programa.
- **Programación lógica:** este paradigma se basa en la idea de usar lógica para representar el conocimiento.
- **Programación declarativa:** este paradigma se centra en describir qué debe hacerse, en lugar de cómo hacerlo.



PARADIGMAS EJEMPLOS

```
# Paradigma Imperativo (en Python)
```

```
suma = 0
for i in range(1, 6):
    suma += i
print("La suma es:", suma)
```

```
# Paradigma Funcional (en Python)
```

```
from functools import reduce

suma_lista = lambda xs: reduce(
    lambda a, b: a + b, xs, 0)

print(suma_lista(range(1, 6)))
```

```
# Paradigma Orientado a Objetos (Python)
```

```
class Calculadora:
    def sumar(self, a, b):
        return a + b

calc = Calculadora()
print("La suma es:", calc.sumar(3, 5))
```

IMPERATIVO O DE PROCEDIMIENTOS

La programación imperativa es un paradigma que se centra en cómo se modifican los estados de un programa a lo largo del tiempo, dando instrucciones a la computadora paso a paso. Se utiliza declaraciones de asignación para asignar valores a variables y declaraciones de control de flujo para determinar el orden de ejecución de las instrucciones.

- **Ventajas**

- Es fácil de entender y de aprender.
- Es versátil y puede ser utilizado para escribir una amplia gama de programas.
- Es rápido y eficiente.

- **Desventajas**

- Puede ser difícil de mantener, especialmente para programas grandes y complejos.
- Puede ser difícil de depurar, especialmente para programas que contienen muchos ciclos.
- Puede no ser adecuado para escribir programas que requieren un alto nivel de concurrencia.



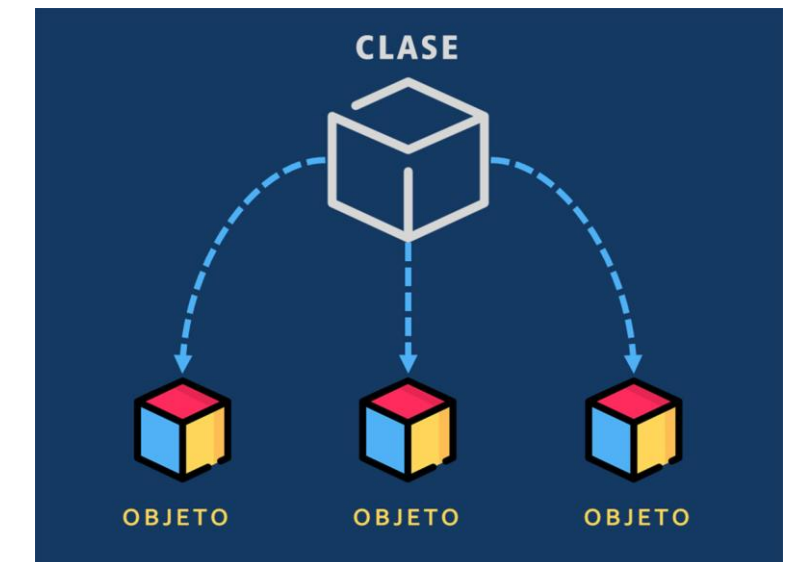
ORIENTADO A OBJETOS



La programación orientada a objetos (OOP) es un paradigma de programación que organiza el código en unidades llamadas objetos. Cada objeto tiene su propio estado y comportamiento. El estado de un objeto se almacena en sus atributos y el comportamiento se define en sus métodos. Los objetos se comunican entre sí enviando mensajes. Un mensaje es una solicitud que un objeto hace a otro objeto para realizar una acción. Los objetos pueden enviar mensajes a sí mismos o a otros objetos.

- **Ventajas**

- **Flexibilidad:** La programación orientada a objetos es muy flexible y puede ser utilizada para escribir una amplia gama de programas.
- **Mantenimiento:** La programación orientada a objetos es muy fácil de mantener, especialmente para programas grandes y complejos.
- **Extensibilidad:** La programación orientada a objetos es muy extensible y puede ser fácilmente ampliada para agregar nuevas funcionalidades.
- **Seguridad:** La programación orientada a objetos es muy segura y puede ayudar a prevenir errores.



PILARES DE POO



- **Abstracción:** Es el proceso de ocultar los detalles de implementación de un objeto y exponer sólo los detalles que son esenciales para su uso. Esto permite que los usuarios de un objeto se concentren en su funcionalidad y no en cómo funciona internamente.
- **Encapsulamiento:** Es el proceso de agrupar juntos los datos y el código que se utilizan para manipular esos datos. Esto permite proteger los datos de acceso no autorizado y hace que el código sea más fácil de entender y mantener.
- **Herencia:** Es el proceso de crear una nueva clase a partir de una clase existente. La nueva clase hereda todas las propiedades y métodos de la clase existente, y puede agregar sus propias propiedades y métodos. Esto permite reutilizar el código y crear clases más complejas más rápidamente.
- **Polimorfismo:** Es la capacidad de una función para tener diferentes significados dependiendo de los argumentos que se le pasen. Esto permite escribir código más genérico y flexible.



FUNCIONAL



La programación funcional es un paradigma de programación que se basa en el uso de funciones. Una función es un bloque de código que toma un conjunto de valores como entrada y devuelve un conjunto de valores como salida. En la programación funcional, las funciones se utilizan para representar el comportamiento de un programa. Las funciones se pueden combinar para crear otras funciones más complejas.

La programación funcional tiene varias ventajas sobre otros paradigmas de programación, como la programación imperativa y la programación orientada a objetos. Algunas de las ventajas de la programación funcional incluyen:

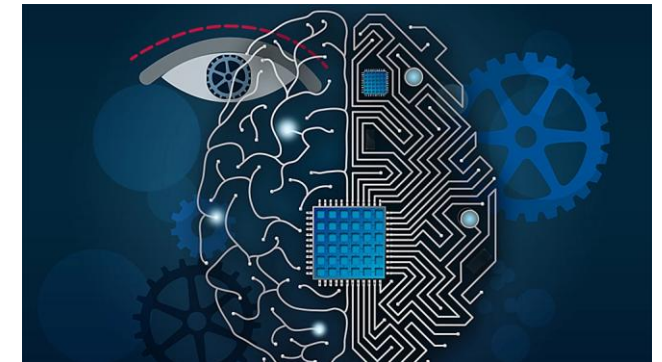
- **Eficiencia:** La programación funcional es muy eficiente y puede ser utilizada para escribir programas que son rápidos y ligeros.
- **Seguridad:** La programación funcional es muy segura y puede ayudar a prevenir errores.
- **Fácil mantenimiento:** La programación funcional es muy fácil de mantener, especialmente para programas grandes y complejos.



La programación lógica es un paradigma de programación que se basa en la lógica matemática. En la programación lógica, los programas se representan como conjuntos de hechos y reglas. Los hechos son declaraciones que son verdaderas, y las reglas son declaraciones que describen cómo se relacionan los hechos.

- **Ventajas**

- **Facilidad de mantenimiento:** Los programas de lógica son muy fáciles de mantener, ya que se pueden modificar fácilmente los hechos y las reglas sin tener que modificar el código.
- **Eficiencia:** Los programas de lógica son muy eficientes, ya que no tienen que realizar operaciones de control de flujo.
- **Verificación:** Los programas de lógica se pueden verificar para garantizar que son correctos.
- **Robustez:** Los programas de lógica son muy robustos y pueden tolerar errores en los datos.



DECLARATIVO



La programación declarativa es un paradigma que se centra en el *qué* se quiere lograr, en lugar del *cómo* hacerlo. En este enfoque, los programadores expresan los resultados deseados mediante declaraciones y expresiones, dejando que el lenguaje de programación se encargue de la ejecución.

Algunas ventajas de la programación declarativa incluyen:

- **Simplicidad:** Permite escribir código más conciso y claro, facilitando la comprensión de lo que se está haciendo.
- **Mantenibilidad:** Hace que el mantenimiento del código sea más fácil, ya que se enfoca en la lógica del programa y no en los detalles de la implementación.
- **Abstracción:** Proporciona un nivel de abstracción que puede hacer que el desarrollo sea más rápido y menos propenso a errores.



CONCEPTOS CLAVE APRENDIDOS

Lenguaje Natural

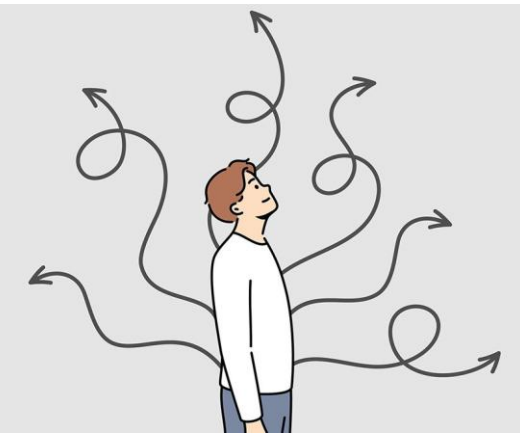
- Usado por los humanos para comunicarse.
 - Ambiguo, flexible y contextual.
 - Ejemplos: Español, Inglés.

Lenguaje Formal

- Conjunto de símbolos y reglas precisas.
 - No ambiguo.
- Usado en matemáticas, lógica y programación.

Lenguajes de Programación

- Sistemas formales diseñados para comunicarse con computadoras.
 - Tienen sintaxis y semántica bien definidas.
 - Ejemplos: Java, Python, C++.



CONCEPTOS CLAVES APRENDIDOS

Evolución de los Lenguajes de Programación

De bajo nivel (ensamblador) a alto nivel (Python, Java).

Mayor abstracción, facilidad de uso y legibilidad.

Incorporación de nuevos paradigmas y características.

Paradigmas de Programación

Imperativo: secuencia de instrucciones (Python, C).

Orientado a Objetos: objetos y clases (Python, Java).

Funcional: funciones puras (Python, Haskell).

Lógico: basado en reglas lógicas (Prolog).



CONCEPTOS CLAVE

APRENDIDOS

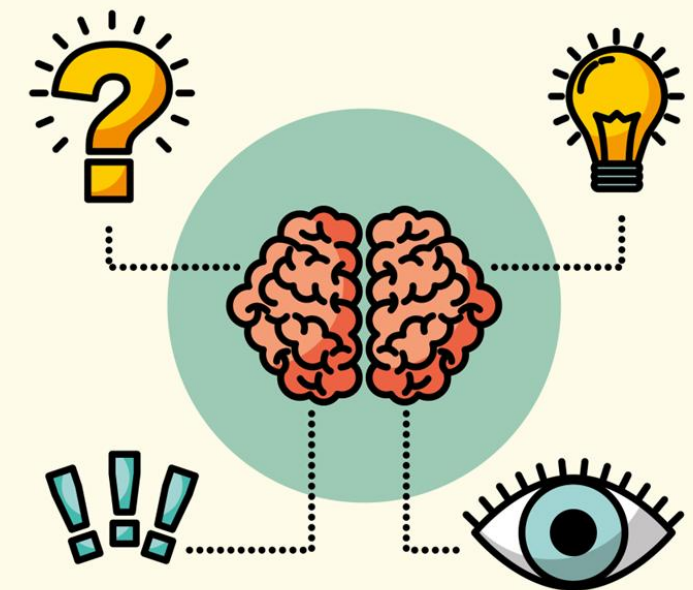
Generaciones de Lenguajes de Programación

- 1ª Generación: lenguaje máquina (binario).
- 2ª Generación: ensamblador.
- 3ª Generación: lenguajes de alto nivel (C, Java).
- 4ª Generación: lenguajes orientados a tareas (SQL).
- 5ª Generación: lenguajes basados en inteligencia artificial y lógica (Prolog).



VALORES

Pensamiento Crítico: Los lenguajes de programación y su evolución no son sólo una cuestión de memorizar reglas, sino también de comprender por qué ciertas decisiones se tomaron en el diseño de lenguajes.



REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.
- SICP. (1996). Structure and Interpretation of Computer Programs (2nd ed.). MIT Press.
- Wirth, N. (1996). Programming in Modula-2 (2nd ed.). Springer.





¡GRACIAS POR SU ATENCIÓN!

DUDAS

**RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER
DUDA QUE TE SURJA EN LA SEMANA**